

Python 数值分析 算法实践



Python Numerical Analysis Algorithm Practice

第13章 数值优化



讲授：XXX



日期：2024年4月19日

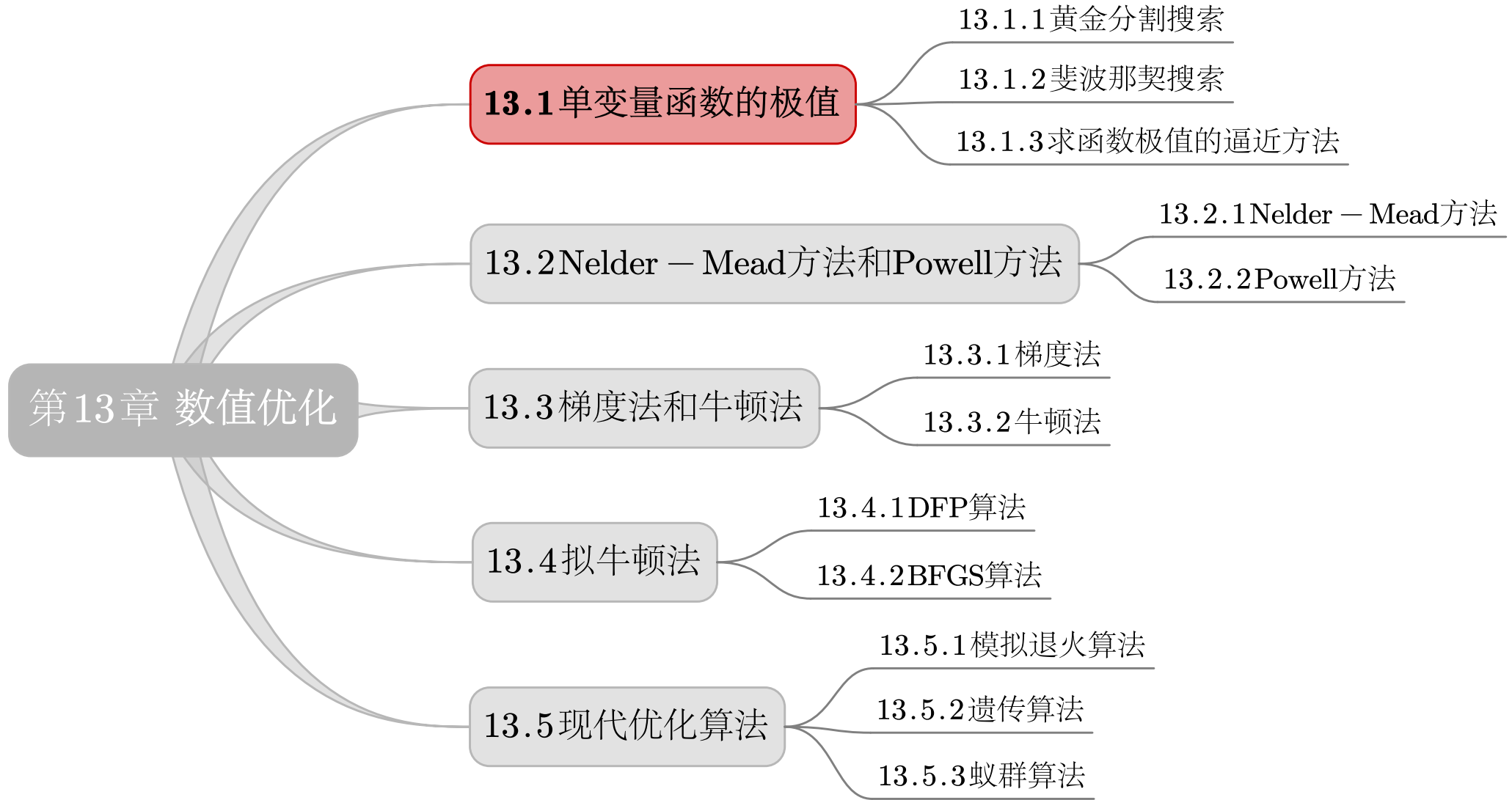
第13章 数值优化

数值优化广泛应用于科学与工程计算、数学建模、数据科学、机器学习、人工智能等领域, 探讨数值优化算法, 有着极其重要的意义. 最优化问题一般可以描述为

$$\min f(\boldsymbol{x})$$

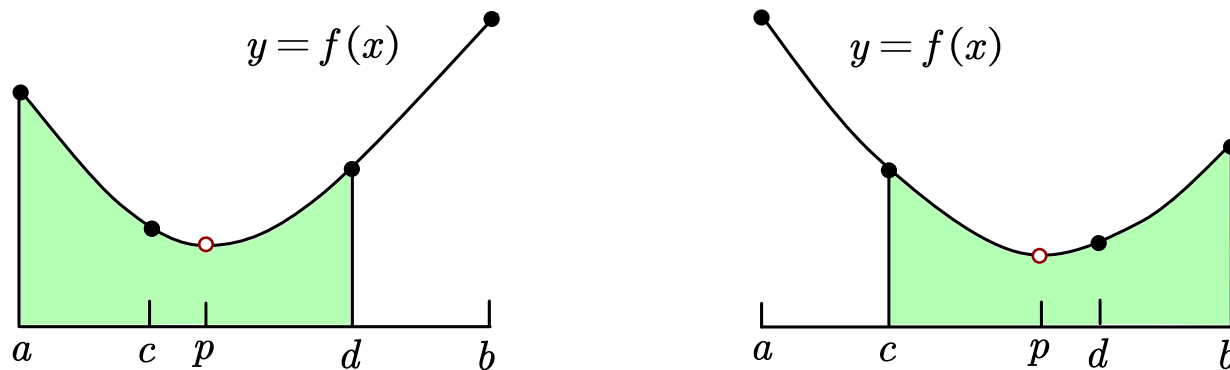
$$s.t. \boldsymbol{x} \in \mathcal{X}$$

其中 $\boldsymbol{x} = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^n$ 为决策变量, $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 是目标函数, $\mathcal{X} \subseteq \mathbb{R}^n$ 是约束集合或可行域. 最优化问题从不同角度可以分为连续和离散优化问题、无约束和约束优化问题、随机和确定性优化问题、线性和非线性规划问题、凸和非凸优化问题等. 本章主要讨论非线性函数的无约束优化问题, 即式(13-1)中 $\mathcal{X} = \mathbb{R}^n$, 对决策变量 $\boldsymbol{x}$ 不加限制, 无需考虑 $\boldsymbol{x}$ 的可行性.



13.1.1 黄金分割搜索

黄金分割搜索算法通过不断压缩区间, 逐步逼近极小值. 分两种情况讨论:



(1) 如果 $f(c) \leq f(d)$, 则从右侧压缩, 使用 $[a, d]$, 左图所示, 并在新区间继续搜索;

(2) 如果 $f(d) < f(c)$, 则从左侧压缩, 使用 $[c, d]$, 右图所示, 并在新区间继续搜索.

选择内点 c 和 d , 使得区间 $[a, c]$ 和 $[d, b]$ 对称, 即 $b - d = c - a$, 其中

$$\begin{cases} c = a + (1 - r)(b - a) = ra + (1 - r)b, \\ d = b - (1 - r)(b - a) = (1 - r)a + rb. \end{cases}$$

通常要求 $r \in (0.5, 1)$ 在每个子区间上保持为常数, 取黄金分割比例, 即 $r = (-1 + \sqrt{5})/2$.

13.1.1 黄金分割搜索

单变量函数极小值的黄金分割搜索法步骤:

- (1) 给定单峰区间 $[a, b]$ 和精度要求 $\varepsilon > 0$;
- (2) 计算 $c = a + (1 - r)(b - a)$, $d = b - (1 - r)(b - a)$;
- (3) 若 $f(c) > f(d)$, 转(4), 否则转(5);
- (4) 若 $d - c < \varepsilon$, 则停止分割与搜索, 极值点为 $(x^*, f(x^*)) = (d, f(d))$; 否则, 令 $a = c$, $c = d$, $d = b - (1 - r)(b - a)$, 转(3);
- (5) 若 $d - c < \varepsilon$, 则停止分割与搜索, 极值点为 $(x^*, f(x^*)) = (c, f(c))$; 否则, 令 $b = d$, $d = c$, $c = a + (1 - r)(b - a)$, 转(3).

若求解的为单变量函数的极大值, 则第(3)步修改为 $f(c) < f(d)$ 即可. 此外, 若无指定的单峰区间, 则可通过进退法寻求目标函数的一个单峰区间.

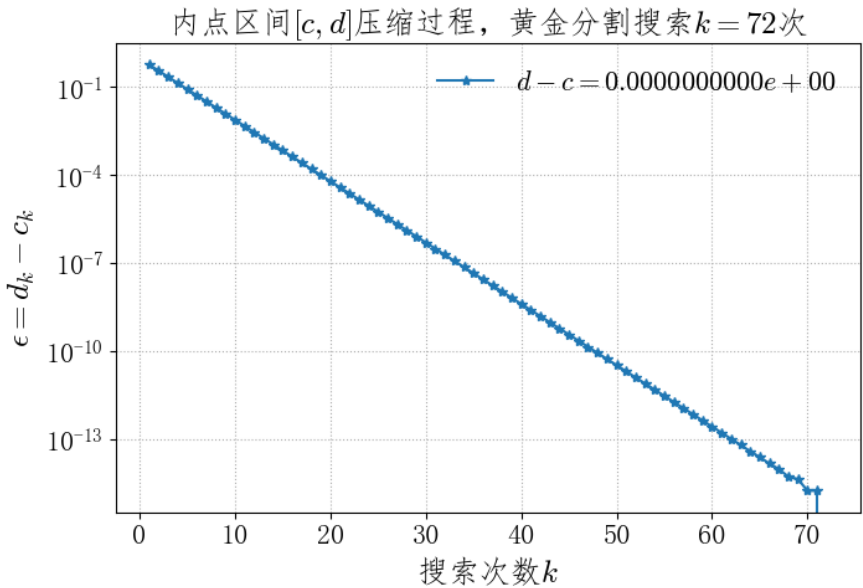
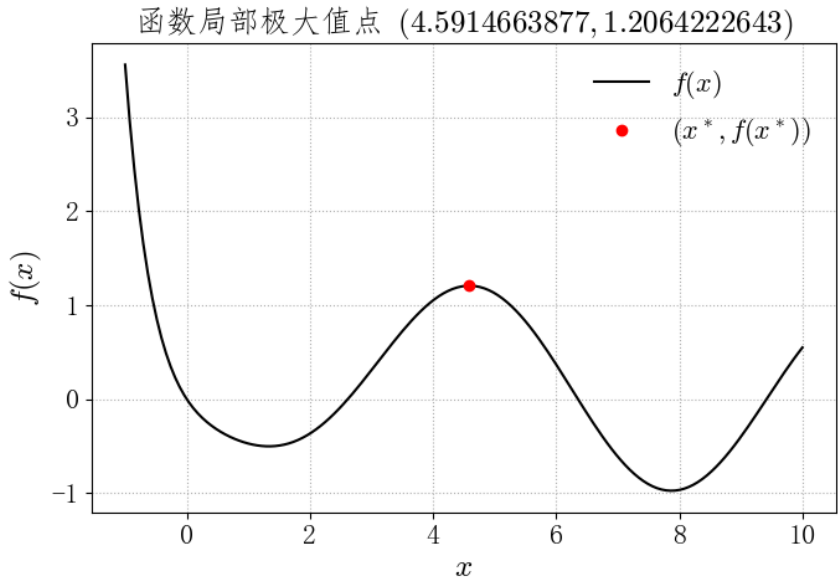
13.1.1 黄金分割搜索

例1:求目标函数 $f(x) = x^2 e^{-x} - \sin x$ 分别在区间 $[0, 2]$ 和区间 $[2, 6]$ 的极小值和极大值.

内点 $d - c$ 的精度要求为 $\varepsilon = 10^{-15}$.

表 13-1 黄金分割搜索法求目标函数极值点及其最终收敛精度

极值类别	x^*	$f(x^*)$	最终内点 $d - c$ 精度
极小值	1.334985936869468	-0.503321534693878	8.881784197e-16
极大值	4.591466387693221	1.206422264276527	0.000000000e+00



13.1.2 斐波那契搜索

斐波那契搜索法与黄金分割搜索法的不同之处在于 r 值不再是常数, 并且子区间数(迭代数)是由指定的容差决定的. 斐波那契搜索是基于公式

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}, n = 2, 3, \dots$$

定义的斐波那契序列 $\{F_k\}_{k=0}^{\infty}$, 即序列可为 $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$.

斐波那契搜索可以从 $r_0 = \frac{F_{n-1}}{F_n}$ 开始, 对 $k = 1, 2, \dots, n-3$ 用 $r_k = \frac{F_{n-1-k}}{F_{n-k}}$, 注意 $r_{n-3} =$

$\frac{F_2}{F_3} = \frac{1}{2}$, 因此这一步无需增加新的点, 故共需 $(n-3) + 1 = n-2$ 步.

将第 k 个子区间的长度按因子 $r_k = \frac{F_{n-1-k}}{F_{n-k}}$ 缩减, 得到第 $k+1$ 个子区间. 最后一个子区间的长度为

$$\frac{F_{n-1}F_{n-2}\cdots F_2}{F_nF_{n-1}\cdots F_1}(b_0 - a_0) = \frac{F_2}{F_n}(b_0 - a_0) = \frac{1}{F_n}(b_0 - a_0) = \frac{b_0 - a_0}{F_n}.$$

13.1.2 斐波那契搜索

如果极小值横坐标的容差为 ε , 则需要找到最小的 n , 使得

$$\frac{b_0 - a_0}{F_n} < \varepsilon \text{ 或 } F_n > \frac{b_0 - a_0}{\varepsilon}.$$

其中 $[a_0, b_0]$ 为原单峰区间. 按照要求, 由公式

$$c_k = a_k + \left(1 - \frac{F_{n-k-1}}{F_{n-k}}\right) (b_k - a_k), \quad d_k = a_k + \frac{F_{n-k-1}}{F_{n-k}} (b_k - a_k).$$

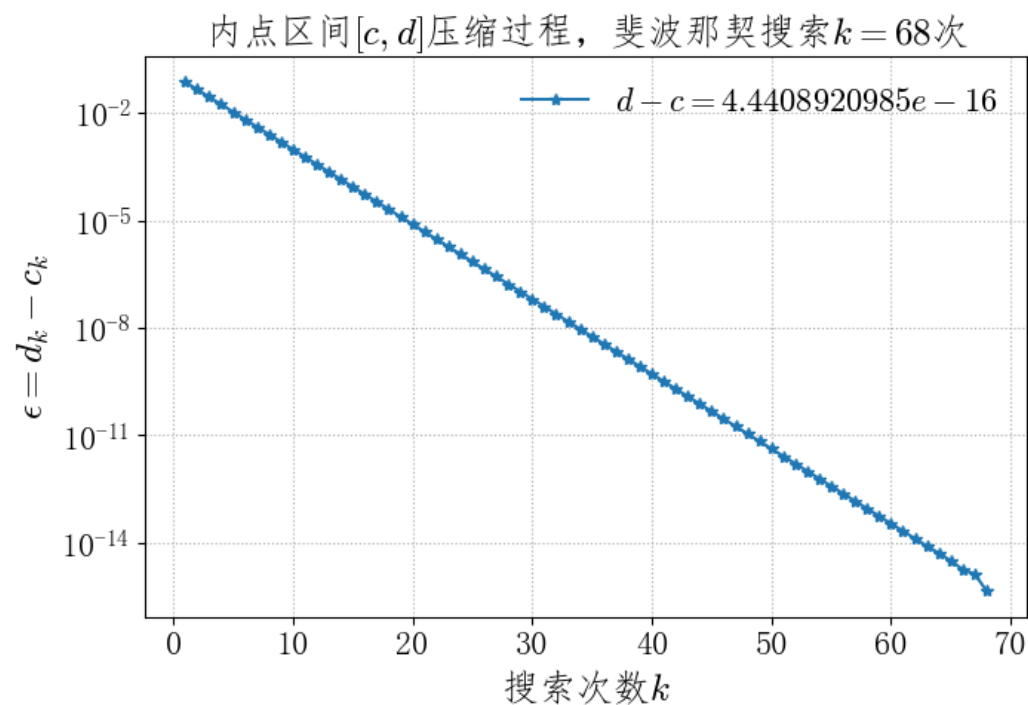
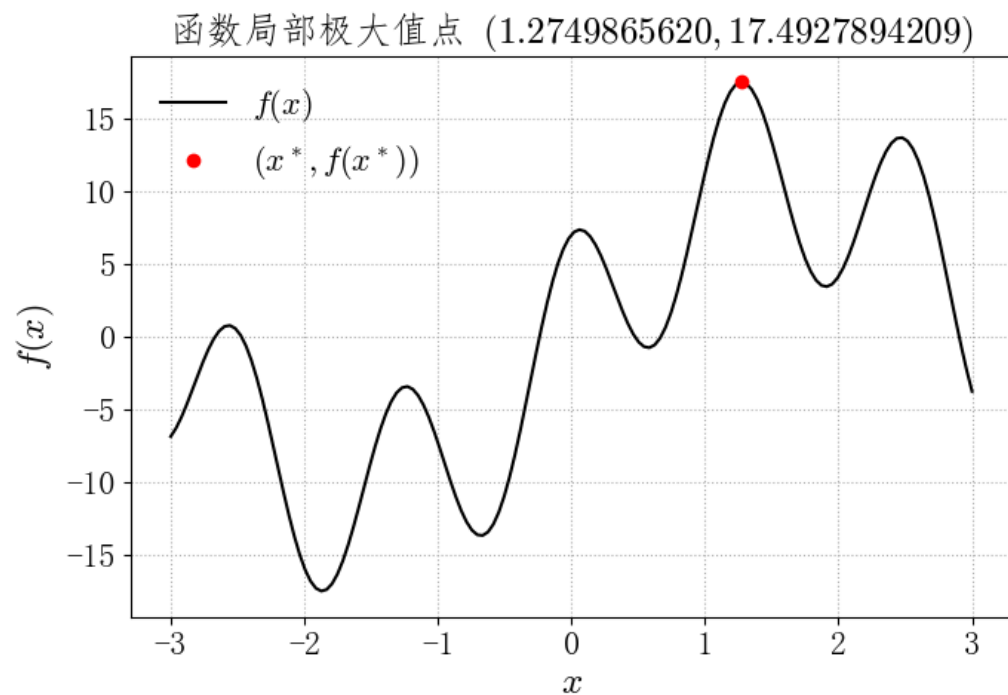
可找到第 k 个子区间 $[a_k, b_k]$ 的内点 c_k 和 d_k .

例2:采用斐波那契搜索法求目标函数 $f(x) = 11\sin x + 7\cos 5x$ 分别在区间 $[-1, 0]$ 和 $[1, 1.5]$ 的极小值和极大值, 精度要求 $\varepsilon = 10^{-15}$.

13.1.2 斐波那契搜索

表 13-2 斐波那契搜索法求目标函数极值问题的结果

极值类别	单峰区间	x^*	$f(x^*)$	斐波那契序列长度
极小值	$[-1, 0]$	-0.677784757828090	-13.684743482837348	75
极大值	$[1, 1.5]$	1.274986562032729	17.492789420899975	74



13.1.3 求函数极值的逼近方法

用二次插值求 p 的近似值 $p_{\min}$, 基于节点 $p_i = p_0 + ih$, $y_i = f(p_i)$, $i = 0, 1, 2$ 的拉格朗日多项式 $L(x)$ 及其一阶导数 $L'(x)$ 为

$$\begin{cases} L(x) = \frac{y_0(x-p_1)(x-p_2)}{2h^2} - \frac{y_1(x-p_0)(x-p_2)}{h^2} + \frac{y_2(x-p_0)(x-p_1)}{2h^2}, \\ L'(x) = \frac{y_0(2x-p_1-p_2)}{2h^2} - \frac{y_1(2x-p_0-p_2)}{h^2} + \frac{y_2(2x-p_0-p_1)}{2h^2}. \end{cases}$$

以 $L'(p_0 + h_{\min})$ 的形式求解 $L'(x) = 0$, 得

$$h_{\min} = \frac{h(4y_1 - 3y_0 - y_2)}{4y_1 - 2y_0 - 2y_2}$$

值 $p_{\min} = p_0 + h_{\min}$ 比 p_0 更好地逼近 p , 以 $p_{\min}$ 替代 p_0 , 并重复计算, 求出新的 h 和 $h_{\min}$. 不断迭代, 直到满足所需精度.

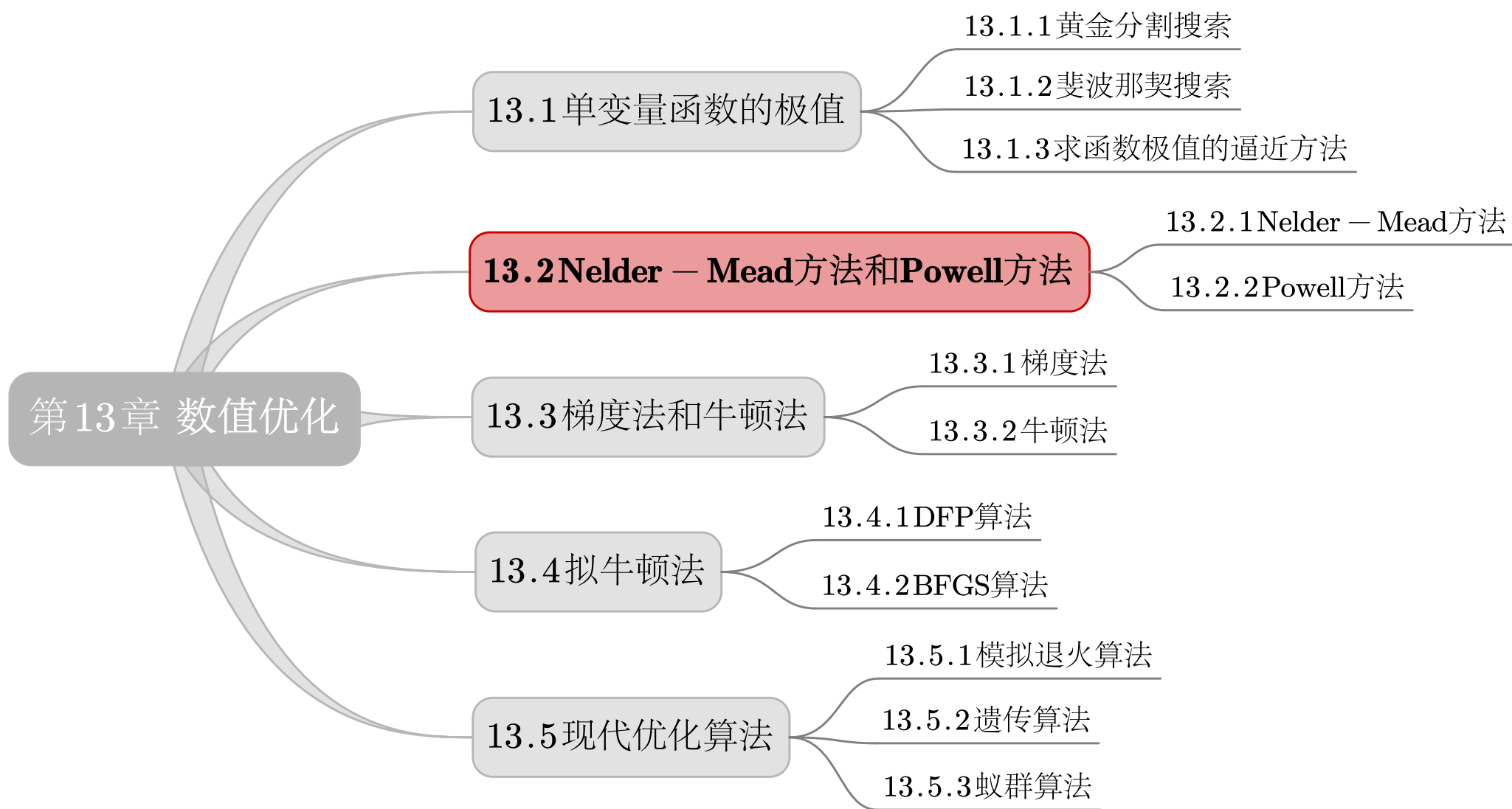
略去求极小值 p 的三次逼近方法.

13.1.3 求函数极值的逼近方法

针对例2示例:分别采用二次插值逼近方法和三次逼近方法求函数 $f(x) = 11\sin x + 7\cos 5x$ 分别在区间 $[-1, 0]$ 和区间 $[1, 1.5]$ 的极小值和极大值. 精度要求为 $\varepsilon = 10^{-15}$.

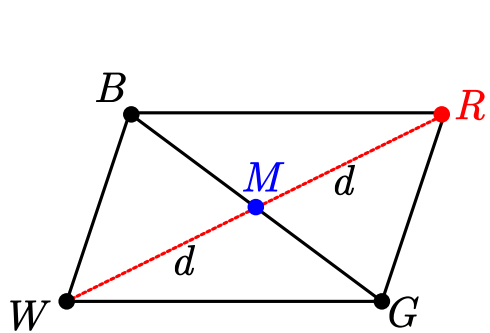
表 13-3 逼近法求解目标函数的极值以及迭代次数

逼近方法	极值类别	x^*	$f(x^*)$	迭代次数
二次插值逼近	极小值	-0.677784755845914	-13.684743482837346	5
	极大值	1.274986564652197	17.492789420899975	6
三次逼近	极小值	-0.677784755781026	-13.684743482837348	7
	极大值	1.274986564409205	17.492789420899975	24

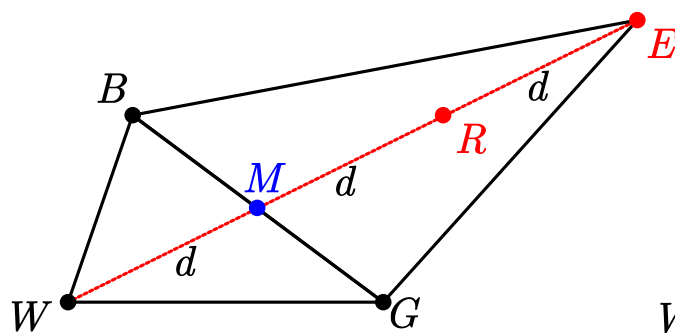


13.2 Nelder-Mead方法和Powell方法

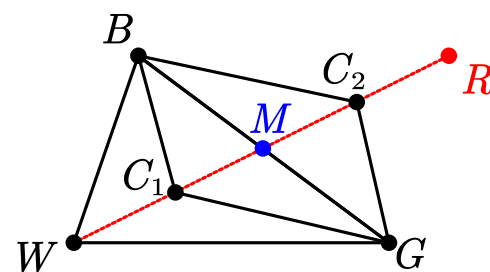
内德(Nelder)和米德(Mead)提出了单纯形(simplex)法, 可用于求解多变量函数的局部极小值. 以二元函数为例, 单纯形记为三角形, 该方法是一个模式搜索过程: 比较三角形三个顶点处的函数值, $f(x,y)$ 值最大的顶点为最差顶点, 用一个新的顶点代替最差顶点, 形成新的三角形, 继续这一过程. 这一过程生成一系列三角形, 函数在其顶点的值越来越小, 进而找到极小值.



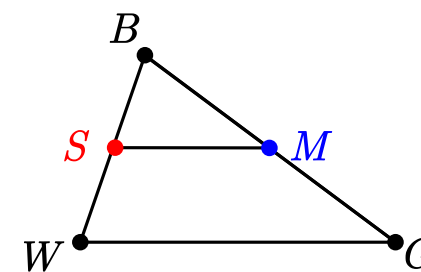
中点 M



反射点 R 和开拓点 E



收缩点 C_1 和 C_2



向 B 收缩三角形

13.2 Nelder-Mead方法和Powell方法

```
IF  $f(R) < f(G)$ , THEN Perform Case(i) {either reflect or extend}  
    ELSE Perform Case(ii) {either contract or shrink}
```

```
BEGIN {Case(i)}
```

```
IF  $f(B) < f(R)$  THEN
```

```
     $W \leftarrow R$ 
```

```
ELSE
```

```
    Compute  $E$  and  $f(E)$ 
```

```
    IF  $f(E) < f(B)$  THEN
```

```
         $W \leftarrow E$ 
```

```
    ELSE
```

```
         $W \leftarrow R$ 
```

```
    ENDIF
```

```
ENDIF
```

```
END {Case(i)}
```

```
BEGIN {Case(ii)}
```

```
IF  $f(R) < f(W)$  THEN
```

```
     $W \leftarrow R$ 
```

```
    Compute  $C = (W + M)/2$  or  $C = (M + R)/2$  and  $f(C)$ 
```

```
    IF  $f(C) < f(W)$  THEN
```

```
         $W \leftarrow C$ 
```

```
    ELSE
```

```
        Compute  $S$  and  $f(S)$ 
```

```
         $W \leftarrow S, G \leftarrow M$ 
```

```
    ENDIF
```

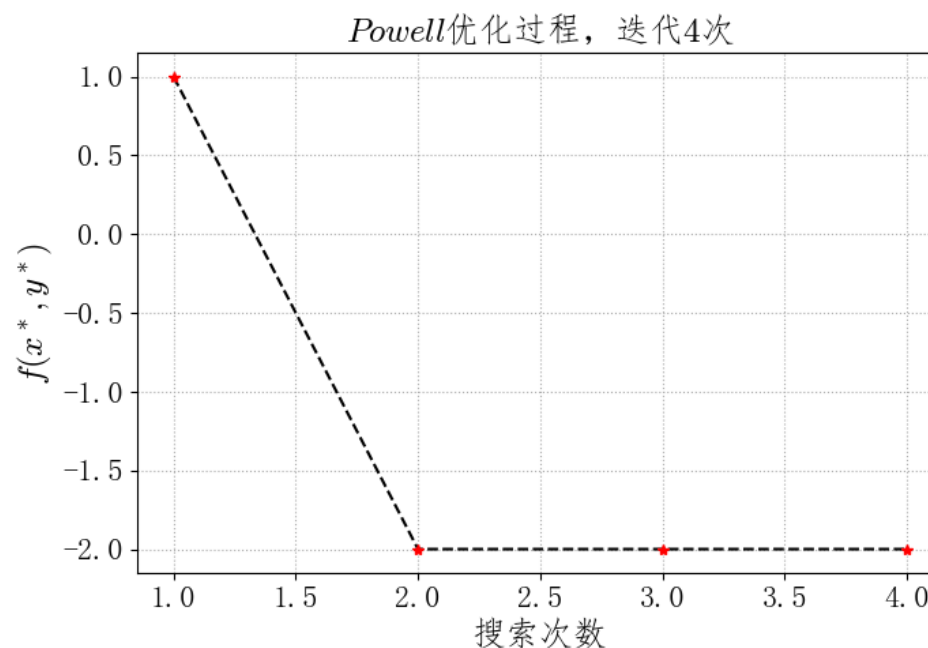
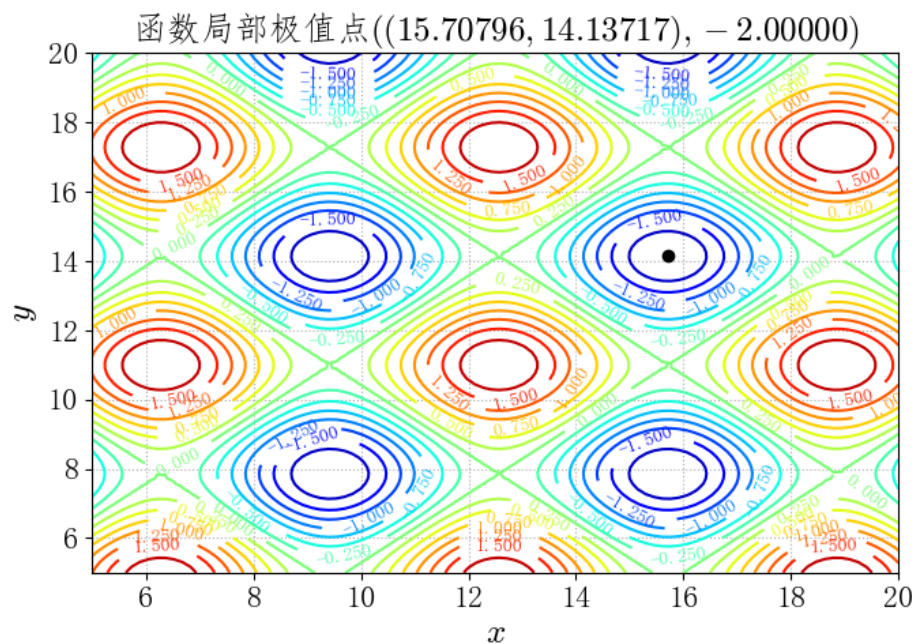
```
END {Case(ii)}
```

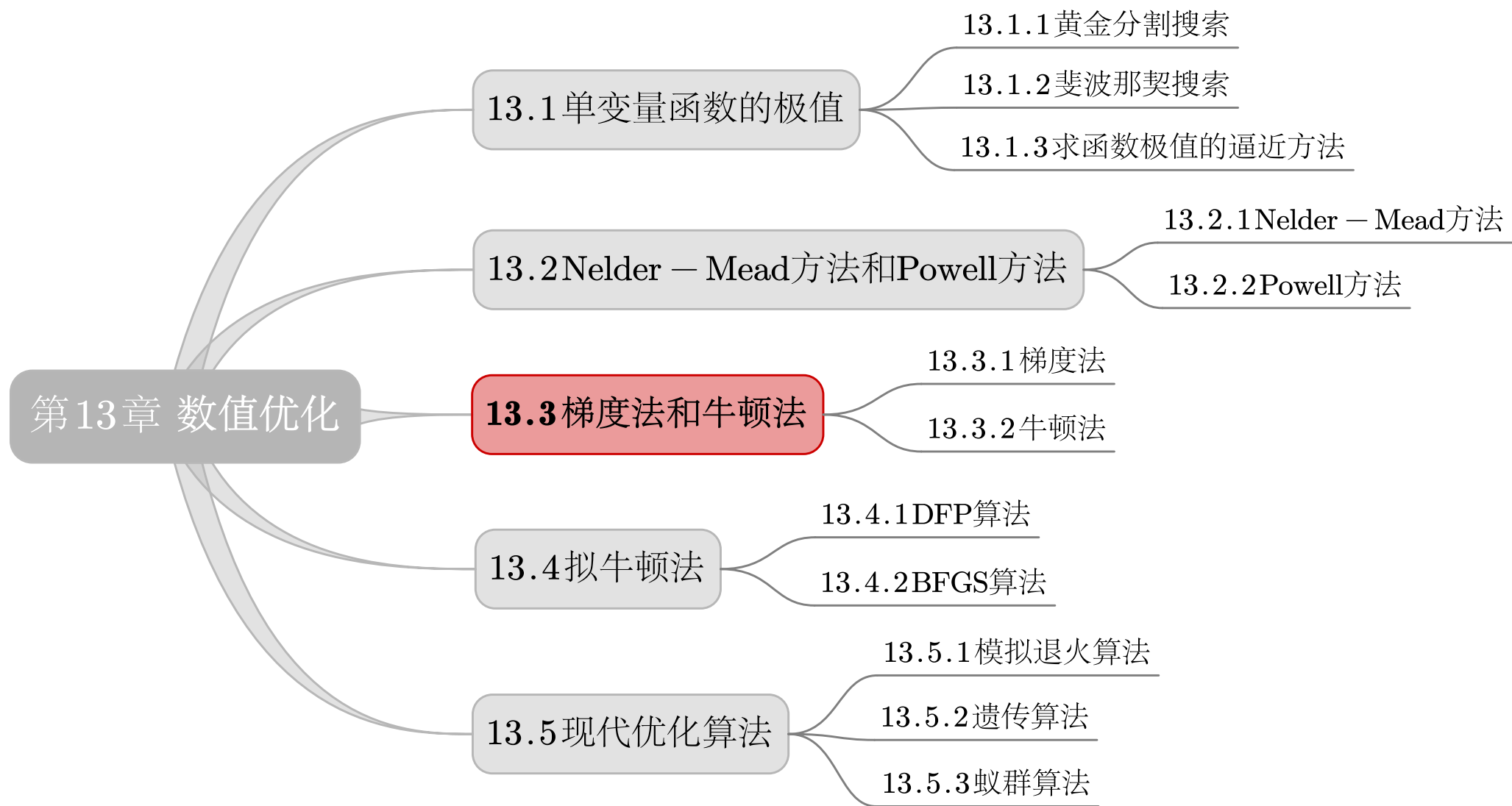
13.2 Nelder-Mead方法和Powell方法

例4:采用鲍威尔算法求二元函数 $f(x,y) = \cos x + \sin y$, $x,y \in [5, 20]$ 的极小值和极大值. 精度 $\varepsilon = 10^{-16}$.

表 13-4 Nelder-Mead 方法求二元目标函数极值结果

极值类别	x_1^*	x_2^*	$f(x_1^*, x_2^*)$	最终收敛精度 ε
极小值	0.611046597235230	-0.305523295760385	-0.641423726326004	8.95090418262362e-16
极大值	-0.937805900553646	0.468902950854937	1.424528382990035	4.440892098500626e-16





13.3.1 梯度法

设 $z = f(\mathbf{X})$ 是 $\mathbf{X} = (x_1, x_2, \dots, x_n)$ 的 $n (\geq 1)$ 元函数, 若偏导数存在, 则 f 的梯度记为 $\nabla f(\mathbf{X})$, 则梯度向量为

$$\nabla f(\mathbf{X}) = \left(\frac{\partial f(\mathbf{X})}{\partial x_1}, \frac{\partial f(\mathbf{X})}{\partial x_2}, \dots, \frac{\partial f(\mathbf{X})}{\partial x_n} \right)$$

梯度向量在局部指向 $f(\mathbf{X})$ 增加最快的方向, 故 $-\nabla f(\mathbf{X})$ 在局部指向 $f(\mathbf{X})$ 下降最快的方向.

设 $\mathbf{P}_k (k=0)$ 为迭代初始点, 梯度法的优化步骤:

(1) 求梯度向量 $\nabla f(\mathbf{P}_k)$.

(2) 计算搜索方向 $\mathbf{S}_k = -\frac{\nabla f(\mathbf{P}_k)}{\|\nabla f(\mathbf{P}_k)\|_2}$.

(3) 在区间 $[0, b]$ 上对 $\Phi(\gamma) = f(\mathbf{P}_k + \gamma \mathbf{S}_k)$ 进行单变量极小化, b 为一个较大值. 这一过程将产生值 $\gamma = h_{\min}$, 它是 $\Phi(\gamma)$ 的一个局部极小值点. 关系式 $\Phi(h_{\min}) = f(\mathbf{P}_k + h_{\min} \mathbf{S}_k)$ 表明, 它是 $f(\mathbf{X})$ 沿搜索线 $\mathbf{X} = \mathbf{P}_k + h_{\min} \mathbf{S}_k$ 的一个极小值.

(4) 构造下一个点 $\mathbf{P}_{k+1} = \mathbf{P}_k + h_{\min} \mathbf{S}_k$.

(5) 精度判断, $\|\mathbf{P}_{k+1} - \mathbf{P}_k\| \leq \varepsilon_1$ 且 $|f(\mathbf{P}_{k+1}) - f(\mathbf{P}_k)| \leq \varepsilon_2$.

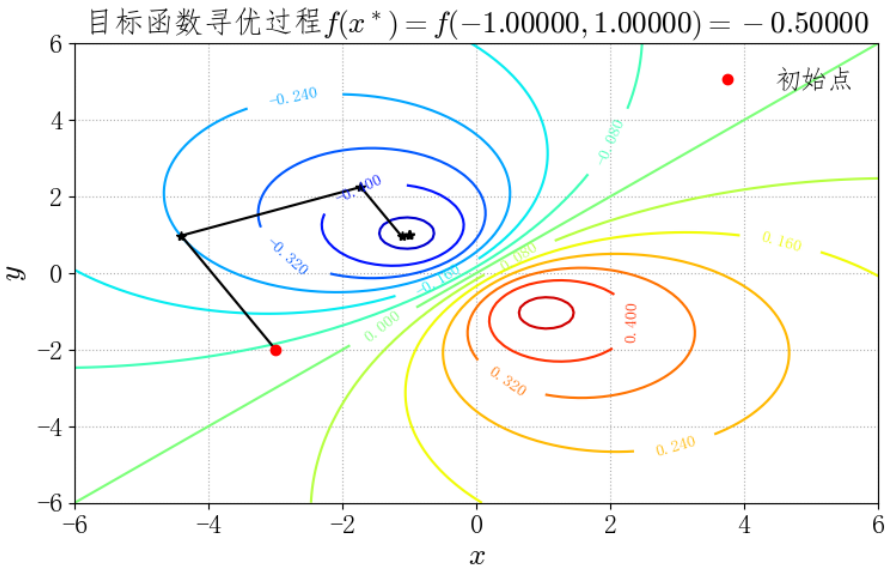
13.3.1 梯度法

例6:用梯度法求函数 $f(x,y)=\frac{x-y}{x^2+y^2+2}$ 的极值, 初始值 $(-3,-2)$ 和 $(0,0)$, 精度要求 $\varepsilon=10^{-16}$.

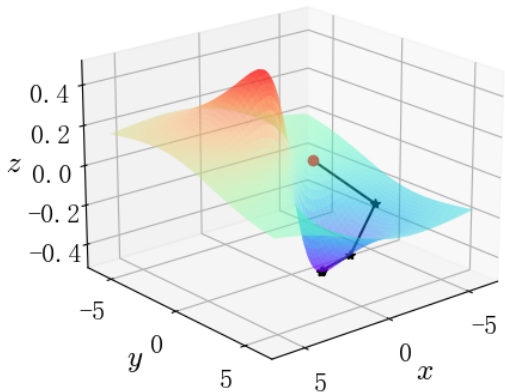
表 13-6 梯度法求解二元目标函数在不同迭代初值下的结果和迭代次数

类型	迭代初值	x^*	y^*	$f(x^*, y^*)$	迭代次数
极小值	$(-3, -2)$	-1.000002695696926	0.999998733466074	-0.499999999998891	7
	$(0, 0)$	-0.999999990215688	0.999999990215688	-0.5000000000000000	3
极大值	$(-3, -2)$	1.000577852431012	-0.999726442172479	0.499999948914363	7
	$(0, 0)$	0.999999990215688	-0.999999990215688	0.5000000000000000	3

梯度法求
二元函数
极小值寻
优过程.



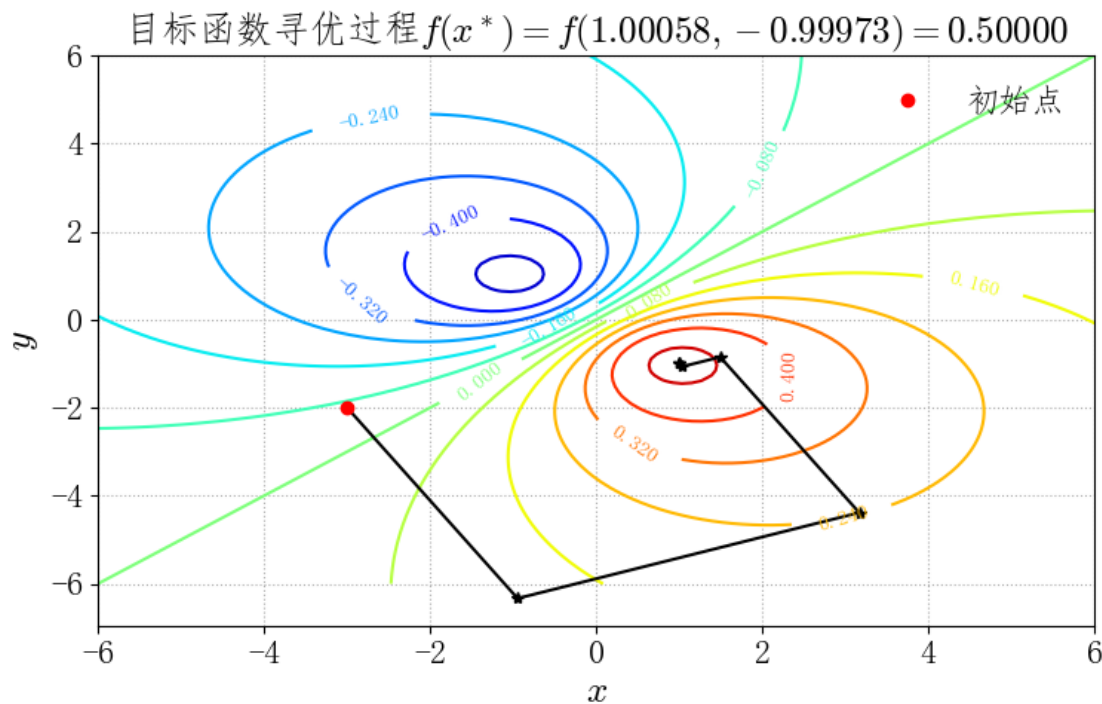
目标函数的梯度下降过程 $x_0=(-3.0, -2.0)$



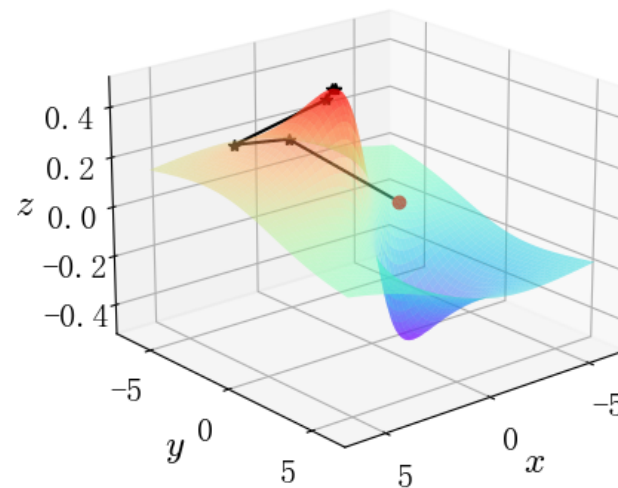
13.3.1 梯度法

例6:用梯度法求函数 $f(x,y) = \frac{x-y}{x^2+y^2+2}$ 的极值, 初始值 $(-3, -2)$ 和 $(0, 0)$, 精度要求 $\varepsilon = 10^{-16}$.

梯度法求二元函数极大值寻优过程.



目标函数的梯度下降过程 $x_0 = (-3.0, -2.0)$



13.3.2 牛顿方法

设 $z = f(\mathbf{X})$ 是 $\mathbf{X} = (x_1, x_2, \dots, x_n)$ 的 $n (\geq 1)$ 元函数, 若一阶和二阶偏导数存在, 则 f 的 Hessian 矩阵记为 $\mathbf{H}f(\mathbf{X})$, 且是一个 $n \times n$ 的对称矩阵

$$\mathbf{H}f(\mathbf{X}) = \begin{bmatrix} \frac{\partial^2 f(\mathbf{X})}{\partial x_1^2} & \frac{\partial^2 f(\mathbf{X})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{X})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(\mathbf{X})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{X})}{\partial x_2^2} & \cdots & \frac{\partial^2 f(\mathbf{X})}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{X})}{\partial x_n \partial x_1} & \frac{\partial^2 f(\mathbf{X})}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{X})}{\partial x_n^2} \end{bmatrix}.$$

13.3.2 牛顿方法

设 $\mathbf{P}_k (k=0)$ 为迭代初始点, 改进的牛顿法的优化步骤:

(1) 计算搜索方向 $\mathbf{S}_k = -\nabla f(\mathbf{P}_{k-1}) ((\mathbf{H}f(\mathbf{P}_{k-1}))^{-1})^T$.

(2) 在区间 $[0, b]$ 上对 $\Phi(\gamma) = f(\mathbf{P}_k + \gamma \mathbf{S}_k)$ 进行单变量极小化, b 为一个较大值. 这一过程将产生值 $\gamma = h_{\min}$, 它是 $\Phi(\gamma)$ 的一个局部极小值点. 关系式 $\Phi(h_{\min}) = f(\mathbf{P}_k + h_{\min} \mathbf{S}_k)$ 表明, 它是 $f(\mathbf{X})$ 沿搜索线 $\mathbf{X} = \mathbf{P}_k + h_{\min} \mathbf{S}_k$ 的一个极小值.

(3) 构造下一个点 $\mathbf{P}_{k+1} = \mathbf{P}_k + h_{\min} \mathbf{S}_k$.

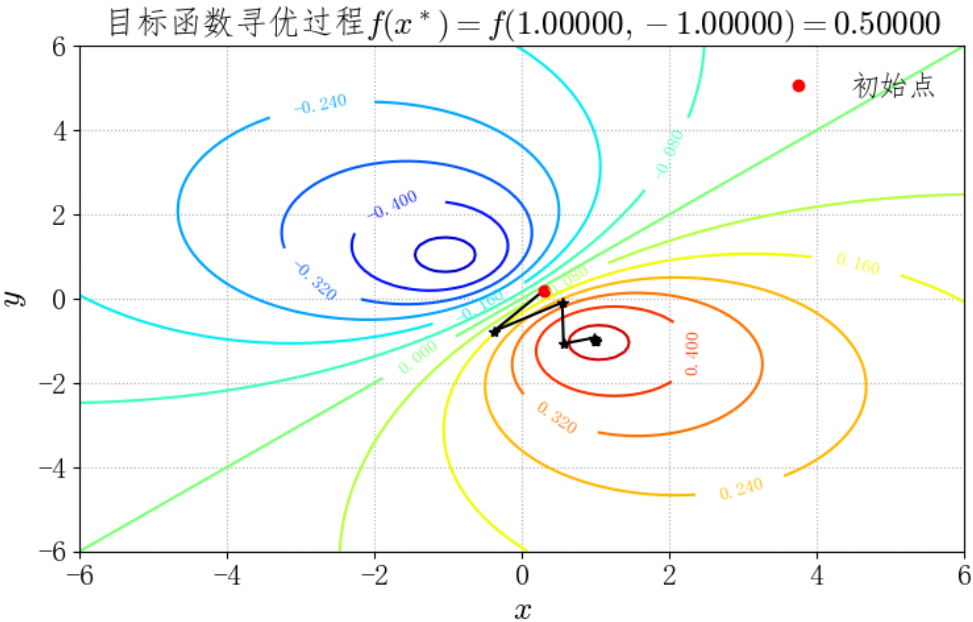
(4) 精度判断, $\|\mathbf{P}_{k+1} - \mathbf{P}_k\| \leq \varepsilon_1$ 且 $|f(\mathbf{P}_{k+1}) - f(\mathbf{P}_k)| \leq \varepsilon_2$.

针对例6 示例:用牛顿法和改进的牛顿法求目标函数 $f(x,y)$ 在初始值 $\boldsymbol{x}_0=(-0.3,0.2)$ 附近的极小值和 $\boldsymbol{x}_0=(0.3,0.2)$ 附近的极大值, 精度要求 $\varepsilon=10^{-16}$.

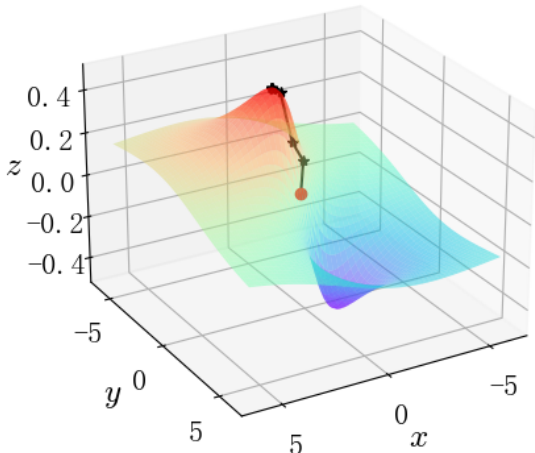
表 13-7 牛顿法与改进的牛顿法求解目标函数极值的结果 (迭代次数中初值除外)

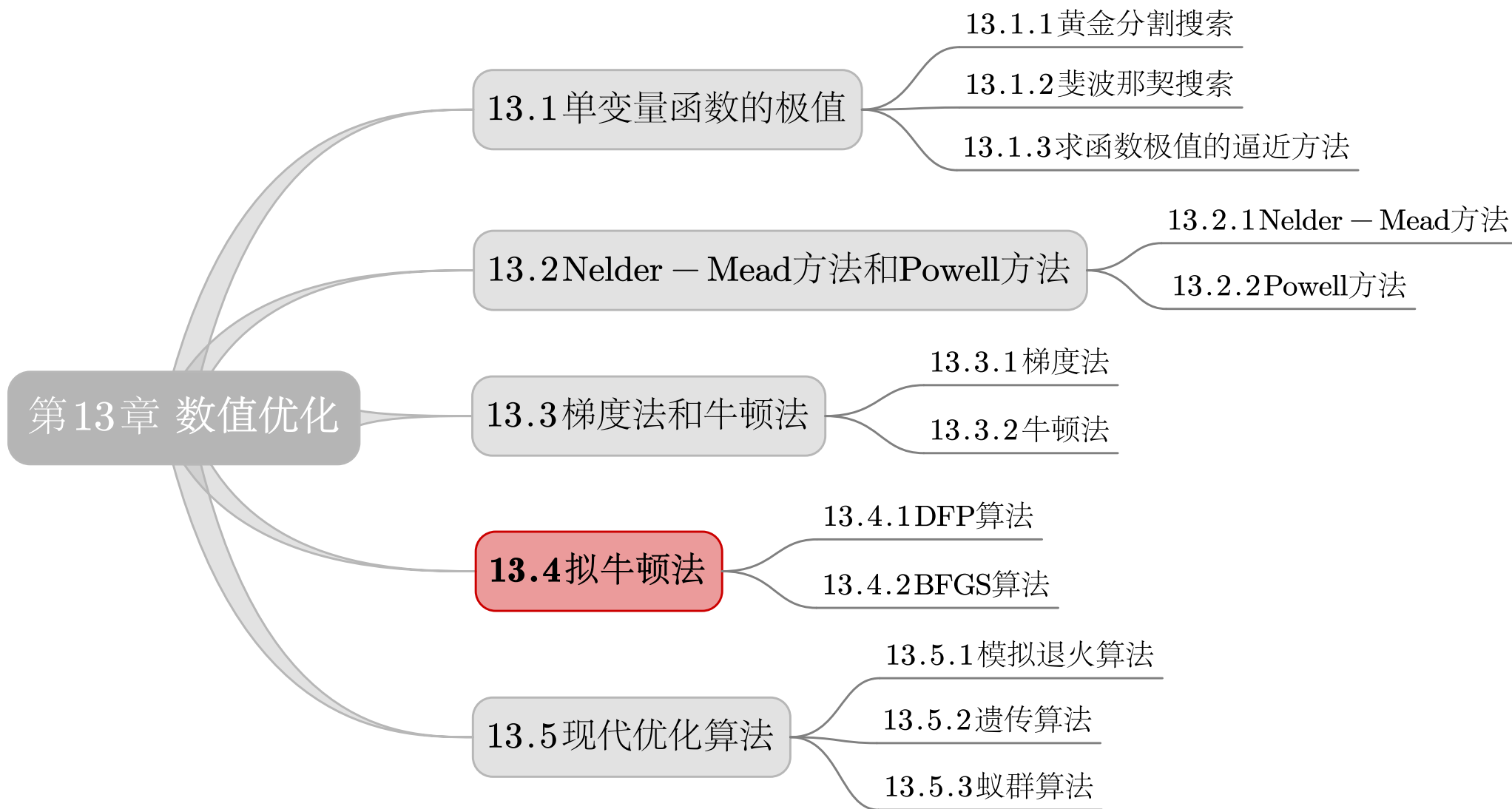
极值类型	优化方法	x^*	y^*	$f(x^*, y^*)$	迭代次数
极小值	改进	-0.999999990910712	1.000000007282156	-0.5000000000000000	6
	非改进	-0.999999999999999	0.999999999999999	-0.5000000000000000	6
极大值	改进	0.999999996667551	-0.999999991667009	0.5000000000000000	10
	非改进	不收敛			

改进的牛
顿法求解
目标函数
的极大值
的寻优过
程 (梯度
下降过程)



目标函数的梯度下降过程 $x_0=(0.3,0.2)$





13.4.1 DFP算法

DFP算法流程：输入目标函数 $f(\mathbf{X})$ ，梯度 $\mathbf{g}(\mathbf{X}) = \nabla f(\mathbf{X})$ ，精度要求 ε 。

(1) 取初始点 $\mathbf{X}^{(0)}$ ，取 $\mathbf{G}_0$ 为正定对称矩阵，置 $k=0$ 。

(2) 计算 $\mathbf{g}_k = \mathbf{g}(\mathbf{X}^{(k)})$ 。若 $\|\mathbf{g}_k\|_2 < \varepsilon$ ，则停止计算，得近似解 $\mathbf{X}^* = \mathbf{X}^{(k)}$ ；否则转(3)。

(3) 置 $\mathbf{p}_k = -\mathbf{G}_k \mathbf{g}_k$ 。

(4) 一维搜索：求使得 $f(\mathbf{X}^{(k)} + \lambda_k \mathbf{p}_k) = \min_{\lambda \geq 0} f(\mathbf{X}^{(k)} + \lambda \mathbf{p}_k)$ 。

(5) 置 $\mathbf{X}^{(k+1)} = \mathbf{X}^{(k)} + \lambda_k \mathbf{p}_k$ 。

(6) 计算 $\mathbf{g}_{k+1} = \mathbf{g}(\mathbf{X}^{(k+1)})$ ，若 $\|\mathbf{g}_{k+1}\|_2 < \varepsilon$ ，则停止计算，得近似解 $\mathbf{X}^* = \mathbf{X}^{(k+1)}$ ；否则，按式

$$\mathbf{G}_{k+1} = \mathbf{G}_k + \mathbf{P}_k + \mathbf{Q}_k = \mathbf{G}_k + \frac{\delta_k \delta_k^T}{\delta_k^T \mathbf{y}_k} - \frac{\mathbf{G}_k \mathbf{y}_k \mathbf{y}_k^T \mathbf{G}_k}{\mathbf{y}_k^T \mathbf{G}_k \mathbf{y}_k}$$

算出 $\mathbf{G}_{k+1}$ 。

(7) 置 $k = k + 1$ ，转(3)。

略去BFGS算法。

13.4.1 DFP算法

针对例3示例：采用DFP算法求 $f(x,y)$ 分别在 $(0,0)$ 和 $(-1,0)$ 附近的极小值和极大值，精度 $\varepsilon = 10^{-16}$.

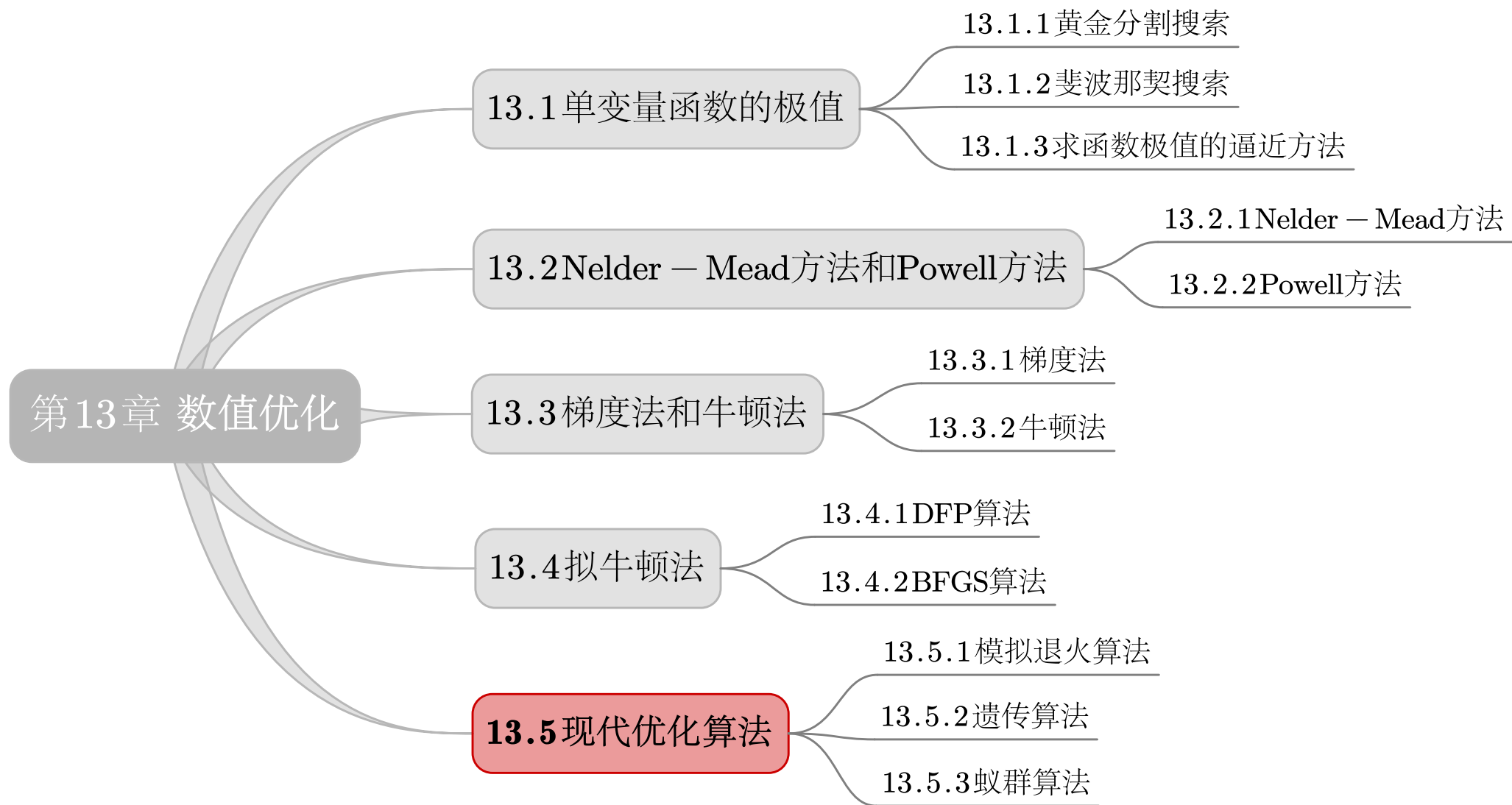
表 13-9 DFP 算法求解目标函数的极值结果

极值类型	x^*	y^*	$f(x^*, y^*)$	迭代次数
极小值	0.611046586125908	-0.305523293147685	-0.641423726326004	7
极大值	-0.937805888608706	0.468902944495155	1.424528382990035	7

针对例6示例：采用DFP算法求 $f(x,y)$ 分别在 $\boldsymbol{x}_0 = (-0.3, 0.2)$ 和 $\boldsymbol{x}_0 = (0.3, 0.2)$ 附近的极小值和极大值，精度 $\varepsilon = 10^{-16}$.

表 13-10 DFP 算法求解目标函数的极值结果

极值类型	x^*	y^*	$f(x^*, y^*)$	迭代次数
极小值	-0.999999993300658	1.000000006706838	-0.5000000000000000	9
极大值	0.999999988432339	-1.000000014749540	0.5000000000000000	7

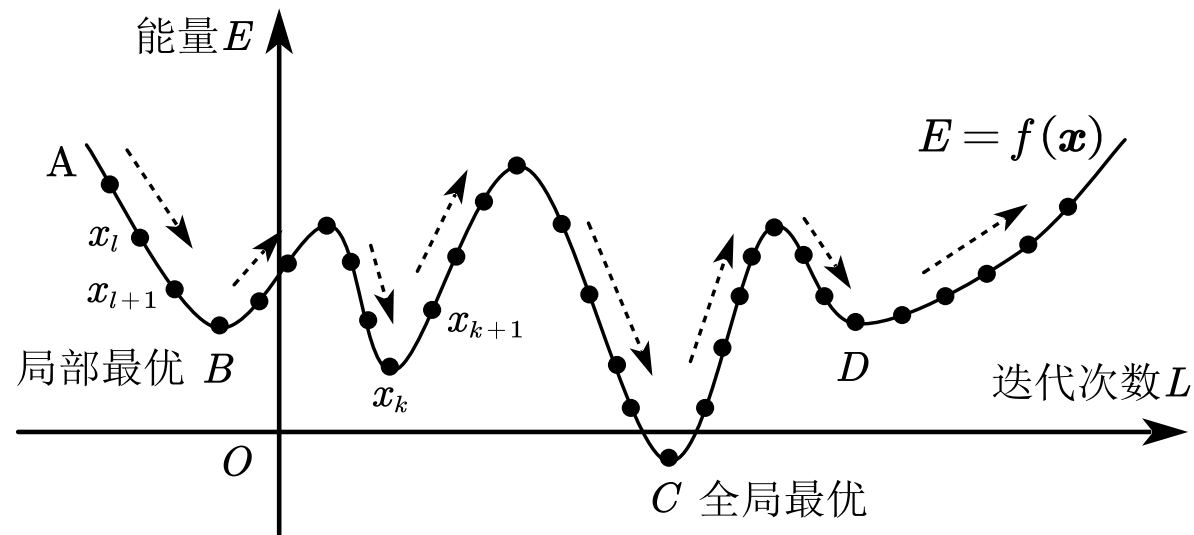


13.5.1 模拟退火算法

如图所示, 在当前温度下, 固体能量不断优化, 直至到达全局最小值. 在寻优过程中, 如果能量处于局部最优 x_k , 则仍会以一定的概率跳出 x_k , 即 $f(x_{k+1}) > f(x_k)$ 当时, 此概率为

$$\exp\left(-\frac{E(k) - E(k+1)}{T_i}\right) = \exp\left(-\frac{f(x_k) - f(x_{k+1})}{T_i}\right)$$

否则, 则以概率 1 接受新解, 如 $f(x_k) < f(x_{k+1})$, 则新解更新 $x_k \leftarrow x_{k+1}$.



13.5.1 模拟退火算法

一般来说, 对于某一个温度 T_i 和该优化问题的一个解 $\mathbf{x}_k$, 可以生成 $\mathbf{x}_{k+1}$. 接受 $\mathbf{x}_{k+1}$ 作为下一个新解的概率为:

$$P(\mathbf{x}_k \rightarrow \mathbf{x}_{k+1}) = \begin{cases} 1, & \text{if } f(\mathbf{x}_{k+1}) < f(\mathbf{x}_k), \\ \exp\left(-\frac{f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k)}{T_i}\right), & \text{otherwise.} \end{cases}$$

在温度 T_i 下, 经过很多次的转移之后, 降低温度 T_i , 得到 $T_{i+1} < T_i$, 在 T_{i+1} 下重复上述过程. 因此整个优化过程就是不断寻找新解和缓慢降温的交替过程. 最终的解是对该问题寻优的结果.

13.5.1 模拟退火算法

新解的产生方法, 即对当前解增加一定的随机扰动, 以便跳出局部最小值. 假设当前解 $\mathbf{x}_k = (x_{k,1}, x_{k,2}, \dots, x_{k,n})$ 和温度 T_i , 首先生成一组服从标准正态分布的随机数 $\mathbf{y} = (y_1, y_2, \dots, y_n)$, 并计算 $\mathbf{z} = \mathbf{y} / \|\mathbf{y}\|_2$, 针对当前解的每一个元素 ($j = 1, 2, \dots, n$), 执行如下操作:

(1) 计算 $x_{k,j}^{new} = x_{k,j} + T_i z_j$, 其中 T_i 是当前温度, 也可使用 $x_{k,j}^{new} = x_{k,j} + \sqrt{T_i} z_j$.

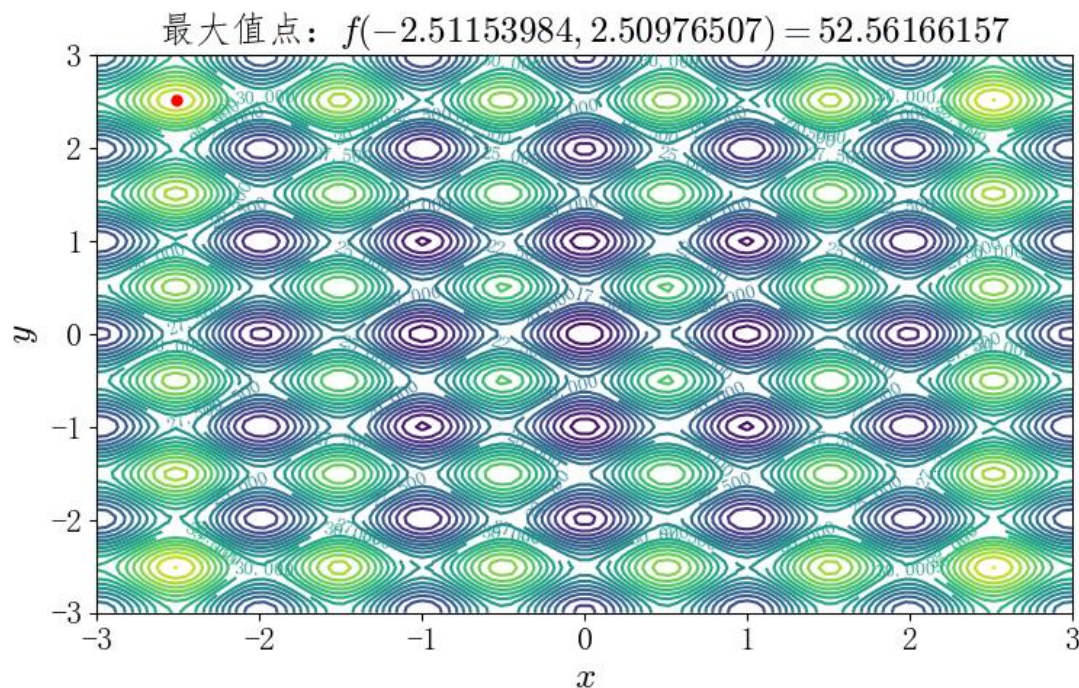
(2) 检查 $x_{k,j}^{new}$ 是否位于上下界 $[l_j, u_j]$, 如果满足 $l_j \leq x_{k,j}^{new} \leq u_j$, 则令 $x_{k+1,j} = x_{k,j}^{new}$. 否则, 如果 $x_{k,j}^{new} < l_j$, 则 $x_{k+1,j} = r \times l_j + (1 - r) \times x_{k,j}$; 如果 $x_{k,j}^{new} > u_j$, 则 $x_{k+1,j} = r \times u_j + (1 - r) \times x_{k,j}$, 其中 $r \sim U(0, 1)$, $[l_j, u_j]$ 为第 j 个变量的求解区间.

略去 13.5.2 遗传算法和 13.5.3 蚁群算法.

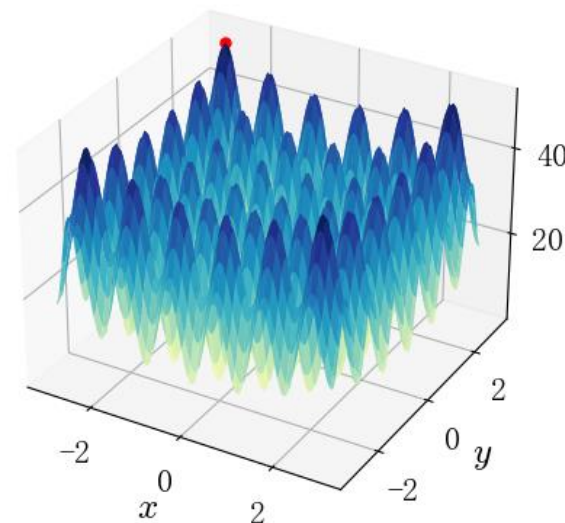
13.5.1 模拟退火算法

例8:求 $f(x, y) = x^2 + y^2 - 10\cos 2\pi x - 10\cos 2\pi y + 20$, $x, y \in [-3, 3]$ 的最小值和最大值.

如图所示, 该目标函数在给定区间存在多个局部极大值, 但最大值分布在等值线图(左图所示)的四个角上(目标函数为偶函数). 由于随机性, 可能每次运行的最大值点标记不同, 但最大值基本一致.



目标函数的三维曲面图及其最值点



13.5.1 模拟退火算法

例8:求 $f(x, y) = x^2 + y^2 - 10\cos 2\pi x - 10\cos 2\pi y + 20$, $x, y \in [-3, 3]$ 的最小值和最大值.

该目标函数也存在多个局部极小值, 但仅有一个最小值, 即点的函数值为0.

